TEMA 7. Aspectos funcionales

1. ¿Qué es un puntero a una función? Pon un ejemplo de código en C, donde se define una función y que reciba una cadena de caracteres como parámetro y devuelva la cadena en mayúsculas. Crea un puntero en una variable local a dicha función llamado `aMayusculas` e invócala con el puntero.

Respuesta

Un puntero a una función en C es una variable que almacena la dirección de memoria de una función. Esto permite invocar la función a través del puntero, lo que resulta útil para pasar funciones como parámetros a otras funciones o para implementar callbacks. En el siguiente ejemplo, se define una función que convierte una cadena de caracteres a mayúsculas y se utiliza un puntero para invocarla:

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

// Función que convierte una cadena a mayúsculas
char* convertirAMayusculas(char* cadena) {
    for (int i = 0; cadena[i] != '\0'; i++) {
        cadena[i] = toupper(cadena[i]);
    }
    return cadena;
}

int main() {
    char texto[] = "hola mundo";

    // Declaración de un puntero a la función
    char* (*aMayusculas)(char*) = convertirAMayusculas;

    // Invocación de la función a través del puntero
    printf("%s\n", aMayusculas(texto));

    return 0;
}
```

En este ejemplo, `aMayusculas` es un puntero a la función `convertirAMayusculas`, y se utiliza para invocar la función con una cadena de entrada.

2. ¿Qué es una **función lambda** en un lenguaje de programación? Pon un ejemplo similar al anterior en Javascript y otro en Java con funciones lambda. Usa una variable local `aMayusculas` para apuntar a la función lambda. Por simplicidad, en Java, emplea

`Function<String, String>` para el tipo de la referencia a la función lambda.

Respuesta

Una función lambda, también conocida como función anónima, es una función que no está ligada a un identificador y que puede definirse de manera concisa en el lugar donde se necesita. Estas funciones son comunes en lenguajes modernos y se utilizan para simplificar el código, especialmente en programación funcional.

En JavaScript, se puede definir una función lambda para convertir una cadena a mayúsculas de la siguiente manera:

```javascript
// Función lambda en JavaScript
const aMayusculas = (cadena) => cadena.toUpperCase();

// Uso de la función
console.log(aMayusculas("hola mundo"));
```

En Java, las funciones lambda se introdujeron en la versión 8. Se puede usar la interfaz funcional `Function` para definir una función lambda que convierta cadenas a mayúsculas:

```java
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        // Función lambda en Java
        Function<String, String> aMayusculas = cadena -> cadena.toUpperCase();

        // Uso de la función
        System.out.println(aMayusculas.apply("hola mundo"));
    }
}
```

En ambos ejemplos, `aMayusculas` es una referencia a la función lambda que realiza la transformación de cadenas a mayúsculas.

3. ¿Qué es el **paradigma funcional**? ¿Por qué a algunos lenguajes orientados a objetos como Java 8, se les llama multi-paradigma? ¿Qué quiere decir que las funciones son "ciudadanos de primera clase"?

Respuesta

El paradigma funcional es un estilo de programación que se centra en el uso de funciones puras y la inmutabilidad de los datos. En este paradigma, las funciones son tratadas como valores, lo que significa que pueden ser asignadas a variables, pasadas como argumentos a otras funciones y devueltas como resultados. Este enfoque promueve un código más declarativo y fácil de razonar.

Java 8 se considera un lenguaje multi-paradigma porque combina características de programación orientada a objetos con elementos de programación funcional. Por ejemplo, Java 8 introdujo las funciones lambda, las interfaces funcionales y las operaciones sobre flujos (`Stream`), lo que permite a los desarrolladores adoptar un estilo funcional cuando sea conveniente, sin abandonar el paradigma orientado a objetos.

El concepto de "funciones como ciudadanos de primera clase" significa que las funciones pueden ser manipuladas como cualquier otro valor en el lenguaje. Esto incluye la capacidad de asignarlas a variables, pasarlas como argumentos a otras funciones y devolverlas como resultados. Este principio es fundamental en la programación funcional y permite una mayor flexibilidad y expresividad en el diseño del software.

4. Explica la sintaxis básica de una función lambda en Java.

Respuesta

La sintaxis básica de una función lambda en Java se introdujo en la versión 8 y sigue el siguiente formato:

```
( parámetros ) -> { cuerpo de la función }
```

- Los **parámetros** son las entradas de la función y se colocan entre paréntesis. Si hay un solo parámetro, los paréntesis son opcionales.

- El **operador** `->` separa los parámetros del cuerpo de la función.
- El **cuerpo de la función** contiene el código que se ejecutará. Si el cuerpo tiene una sola expresión, las llaves `{}` y la palabra clave `return` son opcionales.

Ejemplo de una función lambda que convierte una cadena a mayúsculas:

```
// Función lambda con un solo parámetro y una sola expresión
cadena -> cadena.toUpperCase();

// Función lambda con múltiples parámetros y un cuerpo más complejo
(a, b) -> {
    int suma = a + b;
    return suma;
};
```

En el primer caso, la función toma una cadena como entrada y devuelve la cadena en mayúsculas. En el segundo caso, la función toma dos parámetros, calcula su suma y devuelve el resultado.

5. Ahora recibamos una función como parámetro a un método y la llamaremos desde dentro. Amplia los ejemplos anteriores de Java y JavaScript con un método llamado `transformar`, que reciba un `String` como parámetro y luego una función transformadora como lo es `aMayúsculas` y la invoque desde dentro.

Respuesta

En programación funcional, es común pasar funciones como parámetros a otros métodos. Esto permite aplicar transformaciones o realizar operaciones específicas de manera flexible. A continuación, se amplían los ejemplos en Java y JavaScript con un método `transformar` que reciba una cadena y una función transformadora.

En JavaScript:

```
// Método transformar que recibe una cadena y una función transformadora
function transformar(cadena, transformador) {
    return transformador(cadena);
}

// Función lambda para convertir a mayúsculas
const aMayusculas = (cadena) => cadena.toUpperCase();

// Uso del método transformar
console.log(transformar("hola mundo", aMayusculas));
```

En Java:

```
import java.util.function.Function;

public class Main {
    // Método transformar que reciba una cadena y una función transformadora
    public static String transformar(String cadena, Function<String, String> transformador) {
        return transformador.apply(cadena);
    }

    public static void main(String[] args) {
        // Función lambda para convertir a mayúsculas
        Function<String, String> aMayusculas = cadena -> cadena.toUpperCase();

        // Uso del método transformar
        System.out.println(transformar("hola mundo", aMayusculas));
    }
}
```

En ambos casos, el método `transformar` recibe una cadena y una función transformadora, lo que permite aplicar diferentes transformaciones según sea necesario.

6. Ahora, invoca `transformar`, con una nueva función lambda directamente en la llamada a `transformar`, por ejemplo, una función lambda que invierta la cadena.

Define la función de inversión justo cuando la estás pasando como parámetro.

Respuesta

Es posible definir funciones lambda directamente en la llamada a un método, lo que resulta útil para operaciones específicas que no necesitan ser reutilizadas. A continuación, se muestra cómo invocar `transformar` con una función lambda que invierta una cadena.

En JavaScript:

```
// Uso de transformar con una función lambda para invertir la cadena
console.log(transformar("hola mundo", (cadena) => cadena.split('').reverse().join('')));
```

En Java:

```
public class Main {
    public static void main(String[] args) {
        // Uso de transformar con una función lambda para invertir la cadena
        System.out.println(transformar("hola mundo", cadena -> new StringBuilder(cadena).reverse
    }
}
```

En estos ejemplos, la función lambda que invierte la cadena se define directamente en la llamada al método `transformar`.

7. ¿Qué se entiende por cierre o "closure" en el contexto de las funciones lambda? Pon un ejemplo en Java de cómo una función lambda es capaz de acceder a una variable local en el contexto donde fue definida. Modifica el ejemplo anterior, creando otra función lambda para transformar una cadena, pero que lo que haga es concatenar a la cadena de entrada

otra cadena que está en una variable local definida fuera de la función lambda.

Respuesta

Un cierre o "closure" es una función que captura y recuerda las variables del entorno donde fue creada, incluso después de que dicho entorno haya dejado de estar activo. En el caso de las funciones lambda, esto significa que pueden acceder a variables locales definidas en su contexto de creación.

En Java, las funciones lambda pueden acceder a variables locales siempre que estas sean efectivamente finales (es decir, que no cambien después de su asignación inicial). A continuación, se muestra un ejemplo:

```
public class Main {
    public static void main(String[] args) {
        String sufijo = "!!!"; // Variable local

        // Función lambda que concatena el sufijo
        Function<String, String> agregarSufijo = cadena -> cadena + sufijo;

        // Uso de la función lambda
        System.out.println(transformar("hola mundo", agregarSufijo));
    }
}
```

En este ejemplo, la función lambda `agregarSufijo` accede a la variable local `sufijo` y la utiliza para concatenar su valor a la cadena de entrada.

8. Reflexiona: ¿en qué se diferencia entonces una función lambda de los punteros a funciones que hay en C?

Respuesta

Aunque las funciones lambda y los punteros a funciones tienen similitudes, como la capacidad de referenciar y ejecutar funciones, existen diferencias clave entre ambos conceptos:

1. **Contexto y alcance:** Las funciones lambda pueden capturar variables del contexto donde se definen, lo que las convierte en cierres. Por el contrario, los punteros a funciones en C no tienen esta capacidad; solo almacenan la dirección de una función.
2. **Tipo y seguridad:** En lenguajes como Java, las funciones lambda tienen un tipo bien definido gracias al sistema de tipos estáticos. En C, los punteros a funciones son menos seguros, ya que no hay comprobaciones estrictas de tipos al asignar o invocar funciones.
3. **Sintaxis y expresividad:** Las funciones lambda suelen tener una sintaxis más concisa y expresiva, lo que facilita su uso en programación funcional. Los punteros a funciones en C requieren más código y son menos flexibles.

En resumen, las funciones lambda son más avanzadas y ofrecen características adicionales, como cierres y tipado estático, que no están disponibles en los punteros a funciones de C.

9. Devolvamos ahora funciones. Creemos ahora una función que sea capaz de crear funciones "descuento". Una función "descuento", decrementa un porcentaje pasado como parámetro. Por simplicidad, usa `Function<Double, Double>` para su tipo. La función `crearDescuento(porcentaje)`, recibe solo el porcentaje de descuento a aplicar y devuelve la función de descuento. Prueba a crear dos descuentos distintos y aplicarlos a una cantidad. Explica la closure en la función descuento.

Respuesta

En Java, es posible devolver funciones desde un método. Esto se logra utilizando interfaces funcionales como `Function`. A continuación, se muestra cómo crear una función `crearDescuento` que devuelve una función de descuento:

```java
import java.util.function.Function;

public class Main {
    // Método que devuelve una función de descuento
    public static Function<Double, Double> crearDescuento(double porcentaje) {
        return cantidad -> cantidad - (cantidad * porcentaje / 100);
    }

    public static void main(String[] args) {
        // Crear dos funciones de descuento
        Function<Double, Double> descuento10 = crearDescuento(10);
        Function<Double, Double> descuento20 = crearDescuento(20);

        // Aplicar los descuentos a una cantidad
        System.out.println(descuento10.apply(100.0)); // Resultado: 90.0
        System.out.println(descuento20.apply(100.0)); // Resultado: 80.0
    }
}
```

En este ejemplo, la función `crearDescuento` utiliza una variable local (`porcentaje`) que es capturada por la función lambda devuelta. Esto es un ejemplo de closure, ya que la función lambda recuerda el valor de `porcentaje` incluso después de que el método `crearDescuento` haya terminado su ejecución.

10. En Java, que es un lenguaje con comprobación estática de tipos, donde los tipos se declaran, toda función lambda tiene un tipo, que se conoce como **interfaz funcional**. ¿Qué es una **interfaz funcional**? ¿Qué requisitos tiene?

Respuesta

Una interfaz funcional en Java es una interfaz que tiene exactamente un método abstracto. Estas interfaces se utilizan como tipos para funciones lambda, ya que una función lambda puede implementarlas de manera implícita.

Los requisitos de una interfaz funcional son:

1. **Un único método abstracto:** La interfaz debe tener exactamente un método abstracto. Este método define el contrato que la función lambda debe cumplir.
2. **Anotación `@FunctionalInterface` (opcional):** Aunque no es obligatoria, se recomienda usar esta anotación para indicar que la interfaz está destinada a ser funcional. Esto también permite que el compilador verifique que la interfaz cumple con los requisitos.

Ejemplo de una interfaz funcional:

```
@FunctionalInterface
public interface Operacion {
    int ejecutar(int a, int b);
}
```

Esta interfaz puede ser implementada por una función lambda que realice una operación entre dos enteros.

11. Creemos una interfaz funcional a mano. Por ejemplo, define la interfaz funcional del ejemplo que transforma la cadena en otra. Llámale `Transformador`, que define una función que convierte una cadena de texto (`String`) en otra (`String`).

Respuesta

A continuación, se define la interfaz funcional `Transformador` que transforma una cadena en otra:

```
@FunctionalInterface
public interface Transformador {
    String transformar(String cadena);
}
```

Esta interfaz puede ser utilizada con una función lambda para realizar transformaciones en cadenas. Ejemplo de uso:

```java
public class Main {
    public static void main(String[] args) {
        Transformador aMayusculas = cadena -> cadena.toUpperCase();
        System.out.println(aMayusculas.transformar("hola mundo"));
    }
}
```

12. Ahora hagamos la interfaz funcional algo más genérica y empleando generics, para que permita definir un `Transformador` de un tipo en otro. Pon un ejemplo de un transformador que redondea un `Double` en un `Integer`.

Respuesta

La interfaz funcional `Transformador` puede hacerse genérica utilizando generics. Esto permite transformar un tipo en otro:

```java
@FunctionalInterface
public interface Transformador<T, R> {
    R transformar(T entrada);
}
```

Ejemplo de un transformador que redondea un `Double` a un `Integer`:

```java
public class Main {
    public static void main(String[] args) {
        Transformador<Double, Integer> redondear = valor -> (int) Math.round(valor);
        System.out.println(redondear.transformar(3.14)); // Resultado: 3
    }
}
```

13. `Transformador`, en su versión genérica, parece muy útil y reutilizable, hasta el punto de que es igual a una interfaz funcional que ya hay, que es `Function<T, R>`.

Muestra las interfaces funcionales predefinidas que hay en Java.

Respuesta

Java proporciona varias interfaces funcionales predefinidas en el paquete `java.util.function`. Algunas de las más comunes son:

1. **`Function<T, R>`:** Representa una función que toma un argumento de tipo `T` y devuelve un resultado de tipo `R`.
2. **`Consumer<T>`:** Representa una operación que toma un argumento de tipo `T` y no devuelve resultado.
3. **`Supplier<T>`:** Representa una función que no toma argumentos y devuelve un resultado de tipo `T`.
4. **`Predicate<T>`:** Representa una función que toma un argumento de tipo `T` y devuelve un booleano.
5. **`BiFunction<T, U, R>`:** Representa una función que toma dos argumentos de tipos `T` y `U` y devuelve un resultado de tipo `R`.

Ejemplo de uso de `Function`:

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<String, Integer> longitud = cadena -> cadena.length();
        System.out.println(longitud.apply("hola")); // Resultado: 4
    }
}
```

14. Vamos a ver ejemplos expresivos de funcional en Java. Estudiemos el `List.forEach`, como versión funcional del bucle `for`. Emplea el `forEach` para

recorrer una lista de `Integer` y que muestre un mensaje si el entero es positivo.

Respuesta

El método `forEach` permite recorrer colecciones de manera funcional. A continuación, se muestra un ejemplo:

```
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> numeros = Arrays.asList(-1, 2, -3, 4);

        numeros.forEach(numero -> {
            if (numero > 0) {
                System.out.println("Número positivo: " + numero);
            }
        });
    }
}
```

En este ejemplo, se recorre una lista de enteros y se imprime un mensaje para los números positivos.

15. Repasando el tema de genericidad, fíjate en la firma de `forEach`, ¿por qué se usa `Consumer<? super T>` y no `Consumer<T>`? Explica qué significa **PECS**, y explícalo para el caso de mejorar el ejemplo del método `transformar` la hora de definir el tipo de la función transformadora.

Respuesta

La firma de `forEach` utiliza `Consumer<? super T>` en lugar de `Consumer<T>` para permitir mayor flexibilidad en los tipos. Esto se basa en el principio **PECS**:

- **Producer Extends, Consumer Super (PECS):** Si un tipo produce valores, se usa `extends`. Si consume valores, se usa `super`.

En el caso de `forEach`, el consumidor puede aceptar cualquier supertipo de `T`, lo que permite usar funciones que operen en tipos más generales.

Ejemplo aplicado al método `transformar`:

```
public static <T> void transformar(List<? extends T> lista, Consumer<? super T> accion) {
    lista.forEach(accion);
}
```

Esto permite mayor flexibilidad al trabajar con listas y funciones transformadoras.

16. Referencias a métodos. Podemos obtener una referencia a métodos de objetos o clases. Pon un ejemplo en JavaScript y en Java, de una clase `Persona` con un método `saludar`. En el código principal, crea una `Persona` con un nombre, y obtén una referencia a su método `saludar` en una variable local. Invoca `saludar` con esa referencia a su método `saludar`.

Respuesta

En JavaScript:

```javascript
class Persona {
    constructor(nombre) {
        this.nombre = nombre;
    }

    saludar() {
        console.log(`Hola, soy ${this.nombre}`);
    }
}

const persona = new Persona("Juan");
const referenciaSaludar = persona.saludar.bind(persona);
referenciaSaludar();
```

En Java:

```java
public class Persona {
    private String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }

    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }

    public static void main(String[] args) {
        Persona persona = new Persona("Juan");
        Runnable referenciaSaludar = persona::saludar;
        referenciaSaludar.run();
    }
}
```

17. ¿Qué tipos de referencias a método se pueden hacer en Java? Pon un ejemplo de referencia a método estático, a constructor, a método de instancia

de una instancia concreta y a método de instancia sobre cualquier instancia.

Respuesta

En Java, se pueden hacer los siguientes tipos de referencias a métodos:

1. **Referencia a método estático:**

```
Function<String, Integer> parseInt = Integer::parseInt;
```

2. **Referencia a método de instancia de una instancia concreta:**

```
String texto = "hola";
Supplier<String> toUpperCase = texto::toUpperCase;
```

3. **Referencia a método de instancia sobre cualquier instancia:**

```
Function<String, String> toUpperCase = String::toUpperCase;
```

4. **Referencia a constructor:**

```
Supplier<Persona> crearPersona = Persona::new;
```

18. Otro ejemplo expresivo. Ordena una lista de `Persona`, cada persona tiene un nombre y una edad (de tipo entero). Ordena la lista de `Persona` con `Collections.sort`, pasándole como comparador una expresión lambda que compare la edad de ambas personas y si tienen la misma edad, se ordene por orden alfabético del nombre. Crea dos versiones: Una

con la función de comparación hecha manualmente, y otra empleando `Comparator`.

Respuesta

Versión con comparación manual:

```
Collections.sort(lista, (p1, p2) -> {
    if (p1.getEdad() != p2.getEdad()) {
        return Integer.compare(p1.getEdad(), p2.getEdad());
    }
    return p1.getNombre().compareTo(p2.getNombre());
});
```

Versión con `Comparator`:

```
lista.sort(Comparator.comparingInt(Persona::getEdad).thenComparing(Persona::getNombre));
```